

18th July NLP TASK

NLP Skill Task 2

Vectorized Hidden Markov Model (HMM) for POS Tagging

Import Required Libraries

```
import pandas as pd
```

```
import numpy as np
```

```
import os
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.metrics import classification_report
```

CHECK CURRENT WORKING DIRECTORY

```
print("Current Working Directory:\n")
```

```
print(os.getcwd())
```

LOAD DATASET

```
data = pd.read_csv("ner_dataset.csv", encoding="latin1")
```

DISPLAY DATASET

```
print("\nFirst Five Rows\n")
```

```
print(data.head())
```

```
print("\nLast Five Rows\n")  
print(data.tail())
```

```
print("\nDataset Shape")  
print(data.shape)
```

```
print("\nColumn Names")  
print(data.columns)
```

```
print("\nDataset Information")  
print(data.info())
```

```
# CHECK MISSING VALUES
```

```
print("\nMissing Values Before Filling\n")
```

```
print(data.isnull().sum())
```

```
# FILL MISSING SENTENCE NUMBERS
```

```
data = data.ffill()
```

```
print("\nMissing Values After Filling\n")
```

```
print(data.isnull().sum())
```

```
# TOTAL UNIQUE SENTENCES
```

```
print("\nTotal Number of Sentences")
```

```
print(data["Sentence #"].nunique())
```

```
# TOTAL UNIQUE WORDS
```

```
print("\nTotal Number of Words")
```

```
print(data["Word"].nunique())
```

```
# TOTAL UNIQUE POS TAGS
```

```
print("\nTotal Number of POS Tags")
```

```
print(data["POS"].nunique())
```

```
print("\nPOS Tags")
```

```
print(sorted(data["POS"].unique()))
```

```
# CREATE SENTENCES
```

```
grouped = data.groupby("Sentence #")
```

```
sentences = []
```

```
for sentence_number, sentence in grouped:
```

```
    words = list(sentence["Word"])
```

```
    tags = list(sentence["POS"])
```

```
temp = []

for word, tag in zip(words, tags):

    temp.append((word, tag))

sentences.append(temp)

# DISPLAY SENTENCES

print("\nTotal Sentences Created")

print(len(sentences))

print("\nFirst Sentence\n")

print(sentences[0])

print("\nSecond Sentence\n")

print(sentences[1])

print("\nLength of First Sentence")

print(len(sentences[0]))

# DATASET READY
```

```
print("\nPart 1 Completed Successfully")

train_sentences, test_sentences = train_test_split(
    sentences,
    test_size=0.20,
    random_state=42
)
```

```
print("\nTraining Sentences :", len(train_sentences))
print("Testing Sentences :", len(test_sentences))
```

```
# CREATE UNIQUE POS TAGS

tag_set = set()
```

```
for sentence in train_sentences:
```

```
    for word, tag in sentence:
```

```
        tag_set.add(tag)
```

```
tag_list = sorted(list(tag_set))
```

```
print("\nTotal POS Tags :", len(tag_list))
```

```
print(tag_list)
```

```
# CREATE VOCABULARY

vocabulary = set()
```

```
for sentence in train_sentences:
```

```
    for word, tag in sentence:
```

```
        vocabulary.add(word.lower())
```

```
vocabulary = sorted(list(vocabulary))
```

```
print("\nVocabulary Size :", len(vocabulary))
```

```
# CREATE DICTIONARIES
```

```
tag_to_index = {}
```

```
index_to_tag = {}
```

```
for index, tag in enumerate(tag_list):
```

```
    tag_to_index[tag] = index
```

```
    index_to_tag[index] = tag
```

```
word_to_index = {}
```

```
index_to_word = {}
```

```
for index, word in enumerate(vocabulary):
```

```
    word_to_index[word] = index
```

```
index_to_word[index] = word
```

```
print("\nTag Dictionary Created")
```

```
print("Word Dictionary Created")
```

```
# MATRIX SIZE
```

```
num_tags = len(tag_list)
```

```
num_words = len(vocabulary)
```

```
print("\nNumber of Tags :", num_tags)
```

```
print("Number of Words :", num_words)
```

```
# INITIAL PROBABILITY MATRIX
```

```
initial_probability = np.ones(num_tags)
```

```
for sentence in train_sentences:
```

```
    first_tag = sentence[0][1]
```

```
    initial_probability[tag_to_index[first_tag]] += 1
```

```
initial_probability = initial_probability / initial_probability.sum()
```

```
print("\nInitial Probability Matrix")
```

```
print(initial_probability)
```

```
print("\nShape :", initial_probability.shape)
```

```
# TRANSITION PROBABILITY MATRIX
```

```
transition_probability = np.ones((num_tags, num_tags))
```

```
for sentence in train_sentences:
```

```
    for i in range(len(sentence)-1):
```

```
        current_tag = sentence[i][1]
```

```
        next_tag = sentence[i+1][1]
```

```
        row = tag_to_index[current_tag]
```

```
        column = tag_to_index[next_tag]
```

```
        transition_probability[row][column] += 1
```

```
transition_probability = transition_probability / transition_probability.sum(
```

```
    axis=1,
```

```
    keepdims=True
```



```
)
```

```
print("\nTransition Matrix Shape")
```

```
print(transition_probability.shape)
```

```
# EMISSION PROBABILITY MATRIX
```

```
emission_probability = np.ones((num_tags, num_words))
```

```
for sentence in train_sentences:
```

```
    for word, tag in sentence:
```

```
        word = word.lower()
```

```
        if word in word_to_index:
```

```
            row = tag_to_index[tag]
```

```
            column = word_to_index[word]
```

```
            emission_probability[row][column] += 1
```

```
emission_probability = emission_probability / emission_probability.sum(
```

```
    axis=1,
```

```
    keepdims=True
```

```
)
```

```
print("\nEmission Matrix Shape")
```

```
print(emission_probability.shape)
```

```
# DISPLAY SAMPLE VALUES
```

```
print("\nFirst 10 Initial Probabilities")
```

```
print(initial_probability[:10])
```

```
print("\nFirst 5 x 5 Transition Matrix")
```

```
print(transition_probability[:5, :5])
```

```
print("\nFirst 5 x 5 Emission Matrix")
```

```
print(emission_probability[:5, :5])
```

```
print("\nPart 2 Completed Successfully")
```

```
# PART 3 : CONVERT PROBABILITIES TO LOG SPACE
```

```
log_initial = np.log(initial_probability)
```

```
log_transition = np.log(transition_probability)
```

```
log_emission = np.log(emission_probability)
```

```
print("\nConverted Initial Probability to Log Space")
```

```
print("Converted Transition Probability to Log Space")
```

```
print("Converted Emission Probability to Log Space")
```

```
# VECTORISED VITERBI ALGORITHM
```

```
def viterbi(sentence):
```

```
    sentence = [word.lower() for word in sentence]
```

```
    T = len(sentence)
```

```
    N = len(tag_list)
```

```
    # Dynamic Programming Matrix
```

```
    viterbi_matrix = np.full((N, T), -np.inf)
```

```
    # Back Pointer Matrix
```

```
    backpointer = np.zeros((N, T), dtype=int)
```

```
# INITIALIZATION
```

```
first_word = sentence[0]
```

```
if first_word in word_to_index:
```

```
    emission = log_emission[:, word_to_index[first_word]]
```

```
else:
```

```
    emission = np.log(np.ones(N) / num_words)
```

```
viterbi_matrix[:, 0] = log_initial + emission
```

```
# RECURSION
```

```
for t in range(1, T):
```

```
    current_word = sentence[t]
```

```
    if current_word in word_to_index:
```

```
        emission = log_emission[:, word_to_index[current_word]]
```

```
    else:
```

```
        emission = np.log(np.ones(N) / num_words)
```

```
scores = (  
    viterbi_matrix[:, t-1][:, np.newaxis]  
    + log_transition  
)
```

```
backpointer[:, t] = np.argmax(scores, axis=0)
```

```
viterbi_matrix[:, t] = (  
    np.max(scores, axis=0)  
    + emission  
)
```

```
# TERMINATION
```

```
best_last_tag = np.argmax(viterbi_matrix[:, -1])
```

```
best_path = [best_last_tag]
```

```
# BACKTRACKING
```

```
for t in range(T-1, 0, -1):
```

```
    best_last_tag = backpointer[best_last_tag, t]
```

```
    best_path.insert(0, best_last_tag)
```

```
predicted_tags = [index_to_tag[i] for i in best_path]

return predicted_tags


print("\nVectorized Viterbi Algorithm Created Successfully")


# TEST THE FUNCTION


sample_sentence = [word for word, tag in test_sentences[0]]


predicted = viterbi(sample_sentence)


print("\nSample Sentence")


print(sample_sentence)


print("\nPredicted Tags")


print(predicted)


print("\nActual Tags")


print([tag for word, tag in test_sentences[0]])


print("\nPart 3 Completed Successfully")


# PART 4 : PREDICT POS TAGS FOR TEST DATA
```

```
actual_tags = []

predicted_tags = []

print("\nPredicting POS Tags for Test Sentences...\n")

for sentence in test_sentences:

    words = [word for word, tag in sentence]

    actual = [tag for word, tag in sentence]

    predicted = viterbi(words)

    actual_tags.extend(actual)

    predicted_tags.extend(predicted)

print("Prediction Completed Successfully")


# EVALUATION METRICS

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

```
accuracy = accuracy_score(actual_tags, predicted_tags)
```

```
precision = precision_score(  
    actual_tags,  
    predicted_tags,  
    average="weighted",  
    zero_division=0  
)
```

```
recall = recall_score(  
    actual_tags,  
    predicted_tags,  
    average="weighted",  
    zero_division=0  
)
```

```
f1 = f1_score(  
    actual_tags,  
    predicted_tags,  
    average="weighted",  
    zero_division=0  
)
```

```
print("\n=====")  
print("MODEL EVALUATION")  
print("=====")
```



```
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1-Score : {f1:.4f}")
```

```
# CLASSIFICATION REPORT
```

```
print("\n=====")
print("CLASSIFICATION REPORT")
print("=====\\n")
```

```
print(
    classification_report(
        actual_tags,
        predicted_tags,
        zero_division=0
    )
)
```

```
# CONFUSION MATRIX
```

```
cm = confusion_matrix(actual_tags, predicted_tags)
```

```
print("\nConfusion Matrix Shape :")
print(cm.shape)
```

```
print("\nPart 4 Completed Successfully")
```

```
# PART 5 : TEST THE MODEL ON UNSEEN SENTENCES
```

```
print("\n=====")
```

```
print("TESTING ON UNSEEN SENTENCES")
```

```
print("=====")
```

```
test_sentences_new = [
```

```
    "The boy is playing football",
```

```
    "She loves reading books",
```

```
    "Artificial intelligence is transforming education",
```

```
    "Birds fly in the blue sky",
```

```
    "My mother cooked delicious food today"
```

```
]
```

```
# PREDICT POS TAGS
```

```
for i, sentence in enumerate(test_sentences_new, start=1):
```

```
    words = sentence.split()
```

```
    predicted = viterbi(words)
```

```
    print("\n-----")
```

```

print(f"Sentence {i}")
print("-----")

print("Input Sentence:")
print(sentence)

print("\nPredicted POS Tags:")

for word, tag in zip(words, predicted):

    print(f"{word:20} --> {tag}")

# ANALYSIS
print("\n=====")
print("MODEL ANALYSIS")
print("=====")

print("""

```

1. The Hidden Markov Model successfully predicts the Part-of-Speech tags for most common English words.
2. The Initial Probability Matrix estimates the probability of a sentence beginning with each POS tag.
3. The Transition Matrix learns the probability of one POS tag following another POS tag.

4. The Emission Matrix learns the probability of a word occurring for a particular POS tag.
5. Laplace Smoothing prevents zero probabilities for unseen words and unseen tag transitions.
6. Log-space probabilities prevent numerical underflow caused by multiplying many very small probabilities.
7. The Vectorized Viterbi Algorithm efficiently computes the most probable sequence of POS tags using NumPy matrix operations.
8. Unknown words are handled using uniform emission probabilities, allowing the model to make reasonable predictions even for unseen vocabulary.
9. The HMM POS Tagger performs well for structured English sentences but may struggle with ambiguous words because it relies only on transition and emission probabilities.
10. Overall, the Hidden Markov Model provides an efficient probabilistic approach for Part-of-Speech tagging and achieves good accuracy on the test data.

""")

```
print("\n=====")
print("NLP SKILL TASK 2 COMPLETED SUCCESSFULLY")
print("Vectorized Hidden Markov Model with")
print("Viterbi Decoding for POS Tagging")
print("=====")
```